



AUTUMN-2025

CSE 3527- COMPILER

Department of Computer Science and Engineering (CSE)

আন্তর্জাতিক ইসলামী বিশ্ববিদ্যালয় চট্টগ্রাম
الجامعة الإسلامية العالمية شيتا فونغ
International Islamic University Chittagong

Instructor:	Saifur Rahaman (SR) Assistant Professor, Department of CSE, IIUC Ex-Doctoral Research Fellow, MGH, Harvard Medical School, USA Graduate Researcher, Broad Institute of MIT and Harvard, USA Doctoral Fellow, Department of Computer Science, CityU, HK e-mail: saifurcubd@gmail.com , srahaman@iiuc.ac.bd srahaman@mgh.harvard.edu , srahaman2-c@my.cityu.edu.bd Mobile No: +880 1815 646105		
Time:	Saturday, Sunday, Monday, Tuesday, Wednesday		
Place:	Main CSE Building, Extension Building and FAZ		
Semester:	5 th Semester, Section 5AM, 5CM, 5CF		
Credit Hours:	3	Contact Hours:	3
Textbook:	Compiler Principle and Tools (A. V. Aho, R. Sethi, JD Ullman), 2 nd Ed.		
Reference books:	• Compiler Construction theory and practice, W A Barret, R. M. Bates, 2nd Ed. • Compiler principles, technique and tools, Alfred V Aho, Monica S Lam, Ravi Sethi, Jeffrey D. Ullman, 2nd ed, earson, 2007 • Theory and Practices of Compiler Writing, Trembly and Sorensen • Compiler Construction Principle and practice, KC Louden, CENGAGE • Introduction to Automata Theory, Language, and Computation, John E. Hopcroft, Rajeev Matwani, Jeffrey D. Ullman, 2nd Ed. 2005		
Prerequisite	CSE-3609 Theory of Computing		
Course Rationale / Summary:	This is a course to familiarize the students with knowledge and ideas of the construction of computer program translators called compilers. To take this course, students should be able to gain the skills of automata theory which need to design and implementation of the compiler. The competitive and standard programming ability with some mathematical maturity also will be expected; students should have some idea of computer language grammar and computing skills will also be helpful. After studying this course, the students will have in-depth knowledge of the design and implementation of compilers maintaining the standard steps.		
Course objectives:	1) Grasp of compiler construction. 2) To introduce the major concept areas of language translation and compiler design. 3) To enrich the knowledge in various phases of compiler ant its use, code optimization techniques, machine code generation, and use of symbol table. 4) To extend the knowledge of parser and understand the theory and practice of compiler implementation. 5) To provide practical programming skills necessary for constructing a compiler 6) Utilize tools to automate compiler construction. 7) Develop a large, complex, but well-structured software system that implements various phases of a compiler such as the scanner, parser, code generator, and optimizer. 8) Identify the similarities and differences among various parsing techniques and grammar transformation techniques		

Knowledge Profile (WK):	WK3	Engineering fundamentals	A systematic, theory-based formulation of engineering fundamentals required in the engineering discipline			
	WK4	Specialist knowledge	Engineering specialist knowledge that provides theoretical frameworks and bodies of knowledge for the accepted practice areas in the engineering discipline; much is at the forefront of the discipline.			
	WK6	Engineering practice	Knowledge of engineering practice (technology) in the practice areas in the engineering discipline			
Learning outcomes:	Upon successful completion of this course, students will be able to:					
	#	CO Description			Weightage (%)	
	CO1	Understand the modern compiler construction concepts for advanced computer languages			10	
	CO2	Apply the theory of computation for compiler construction			20	
	CO3	Analyze the lexical, syntax, semantic phases and parser of front-end compiler construction			20	
	CO4	Evaluate the different techniques for code generation and code optimization of back-end compiler construction			40	
	CO5	Ability to design a well-structured compiler that implements various compiler phases, such as the scanner, parser, code generator, and optimizer			10	
Mapping of CO-PO:	#	CO Description	POs	Bloom's Taxonomy / DL	WK	WP
	CO1	Understand the modern compiler construction concepts for advanced computer languages	PO1	C2	WK3	P1
	CO2	Apply the theory of computation for compiler construction	PO1	C3	WK3	P1
	CO3	Analyze the lexical, syntax, semantic phases and parser of front-end compiler construction	PO2	C4	WK3	P1
	CO4	Evaluate the different techniques for code generation and code optimization of back-end compiler construction	PO5	C5	WK4, WK6	P1
	CO5	Ability to design a well-structured compiler that implements various compiler phases, such as the scanner, parser, code generator, and optimizer	PO2	C6	WK4	P1

Course Grading:

Class Attendance	-	10 Marks	10 Marks	100 Marks
Class test	-	10 Marks	10 Marks	
Section A (Mid Term)	-	30 Marks	30 Marks	
Section B (Final Exam)	Group A	20 Marks	50 Marks	
	Group B	30 Marks		
Grading Policy:	As per IIUC grading policy			

Student Civility at Classroom:

- In an effort to make this class enjoyable for everybody...
- Please be on time to class!
- Please do not talk to your friends and neighbors in class! It disturbs everyone, and makes it hard to concentrate. If you have a question, just ask me!
- Please turn your pagers and cell-phones off!
- No permission needed to enter the classroom

Lecture Plan

Week	Activities				
	Lectures	Discussion Topics	Segments	Assigned	Due
Section-A (Mid-term: 30 Marks)					
1	L1	Introduction: Class information and Introduction to compiler	S-1		
	L2, L3	Structure of compiler: Translator, Basic principles of compilers and compiler design	S-1		
2	L4, L5	Structure of compiler: types of compilers, applications, phases of a compiler	S-1		
	L6	Review of Segment 1 Class Test – 1	S-1	CT-1	
3	L7, L8	Grammars and Expressions: Concepts of languages and Grammars, Terminals and non-terminals	S-2		
	L9	Grammars and Expressions: Useless non-terminals and Inaccessible tokens			
4	L10	Grammars & Expressions: Context-Sensitive Grammars, right linear grammar, left linear grammar			
4	L11, L12	Grammars & Expressions: Context-Free Grammars Derivations and Parse trees, Ambiguous grammar Review of Segment 2	S-2	A-1	
5	L13	Automata: Lexical analysis, Regular expressions	S-3		
5	L14, L15	finite automata, deterministic Finite-State Automata (DFA), non-deterministic Finite-State Automata (NFA)			
6	L16	Automata: Minimization of Finite-State Automata, Symbol table management			
6	L17	Review of segment 3 Class Test – 2	S-3	CT-2	
6	L18	Discussion Class for Mid-term Exam	S-1 to S-3		
Section-B (Final Exam: 50 Marks)					
Group-A (20 Marks)					
7	L19	Syntax Analysis: Parser, Error-Recovery	S-4		
7	L20, L21	Syntax Analysis: left recursion, left factoring, Top-Down Parsers			
8	L22, L23	Syntax Analysis: FIRST and FOLLOW, Recursive-Descent and Predictive Parsing,	S-4	A-2	
8	L24	Syntax Analysis: LL(1) parsing, Review of Segment 4	S-4		
9	L25, L26	Parsing: Bottom-Up Parsing, Operator Precedence parsing, LR Parsers	S-5		
9	L27	Parsing: Construction of SLR	S-5		
10	L28, L29	Parsing: CLR, LALR Parse Tables, Shift-Reduce Parsing			
10	L30	Review of Segment 5 Class Test – 3	S-5	CT-3	

Group-B (30 Marks)					
11	L31, L32	Syntax-Directed Translation: Syntax-Directed definitions, Semantic Rules, Dependency Graph, Attributes, Construction of Syntax trees	S-6		
11	L33	Syntax-Directed Translation: Type checking, Type expressions, Translation Scheme	S-6		
12	L34, L35	Syntax-Directed Translation: Type checking of expressions, statements and functions, Type conversions, statements and functions, Type conversions Review of Segment 6	S-6	A-3	
12	L36	Intermediate code generation: Run time environment, Activation tree, Control stack	S-7		
13	L37, L38	Intermediate code generation: Run time storage organization, Storage allocation strategies, Intermediate languages, three address statements, quadruples	S-7		
13	L39	Intermediate code generation: triples, indirect triples and polish notations Review of Segment 7	S-7	A-4	
14	L40, L41	Code Generation and Optimization: Code generation issues, Instruction costs, Basic blocks, Flow graphs, DAG representation, Next use information	S-8		
14	L42	Code Generation and Optimization: Transformations of Basic Blocks, Peephole optimization, Function preserving transformations	S-8		
15	L43, L44	Code Generation and Optimization: Optimization of Basic Blocks, Different models of code generation, error detection and error handling	S-8		
15	L45	Discussion Class for Final Exam Class test - 4	S-4 to S-8	CT-4	
Final Examination (November 2024)			Saifur Rahaman Assistant Professor, CSE, IIUC		



AUTUMN-2025

CSE 3528 – COMPILER LAB

Department of Computer Science and Engineering (CSE)

আন্তর্জাতিক ইসলামী বিশ্ববিদ্যালয় চট্টগ্রাম
الجامعة الإسلامية العالمية شيتاغونغ
International Islamic University Chittagong

Credit Hours: 1

Contact Hours: 2 per week

Type: Core, Engineering

Co-requisite: CSE-3527 Compiler

Course Rationale / Summary:

This compiler lab course delves into the practical aspects of compiler construction, equipping students with the skills to build programs that translate code from human-readable source program in high-level languages into target program. It familiarizes the students with hands-on knowledge and practical ideas of the construction of compilers. Students also able to gain the skills to design and implementation of the compiler for the target programming language. The competitive and standard programming ability will be expected; students should have some idea of theoretical compiler design and development skills will also be helpful. After studying this course, the students will have a practical in-depth knowledge of the design and implementation of compilers and ability to develop well-structured compiler that implements various compiler construction tools, such as the scanner, parser, code generator, and optimizer.

Learning objectives:

- To implement the Lexical Analyzer by using various Lexical Analyzer tools and C programming language
- Design and implementation of Syntax Analyzer and parser
- Experiments for NFA, DFA, CFG from given regular expression
- Develop the program to solve complex problems in compiler
- Design and implement the frontend of the compiler by means of generating Intermediate codes.
- To implement code optimization techniques.
- To implement the backend of the compiler
- To provide an Understanding of the language translation peculiarities by designing complete translator for a simple language
- Construction of the compiler for C programming language

Knowledge Profile (WK):

WK3	Engineering fundamentals	A systematic, theory-based formulation of engineering fundamentals required in the engineering discipline
WK4	Specialist knowledge	Engineering specialist knowledge that provides theoretical frameworks and bodies of knowledge for the accepted practice areas in the engineering discipline; much is at the forefront of the discipline.
WK6	Engineering practice	Knowledge of engineering practice (technology) in the practice areas in the engineering discipline

Complex Engineering Problem Solve: P1, P7

Complex Engineering Activities: N/A

Learning Domains:

Cognitive - C3: Applying, C4: Analyzing, C6: Create

Course Outcomes (COs):

Upon successful completion of this course, students will be able to:

#	CO Description	Weightage (%)
CO1	Apply the theoretical knowledge of compiler construction to develop a scanner, tokenizer and parser	10
CO2	Analyze the new tools and technologies to design and develop the modern compiler construction through hands-on experiments	30
CO3	Develop a well-structured compiler that implements various compiler phases, such as the scanner, parser, code generator, and optimizer	60

Mapping of CO-PO:

#	CO Description	POs	Bloom's Taxonomy/ Domain Level	WK	WP
CO1	Apply the theoretical knowledge of compiler construction to develop a scanner, tokenizer and parser	PO1	C3	WK3	P1
CO2	Analyze the new tools and technologies to design and develop the modern compiler construction through hands-on experiments	PO5	C4	WK4	P1
CO3	Develop a well-structured compiler that implements various compiler phases, such as the scanner, parser, code generator, and optimizer	PO2 PO5	C6	WK6	P1, P7

Course Grading:

Assessment Tools		Weightage (%)		
Class Attendance	-	10	10	100
Continuous evaluation and	Assignments	10	40	
	Class performances	30		
	Mid-term exam			
Final Exam	Quiz	10	50	
	Lab report Book	5		
	Lab Experiments	30		
	Sessional Viva	5		
Grading Policy	As per IIUC grading policy			

Weekly Activity Plan:

Activities: Hand-on (LAB), Assignment (A), Evolution (E), Report (R), and Mid-term Exam (MT)

Week	Activities				
	LAB		Name of Experiments	Assigned	Evaluation
1	L1		Introduction to compiler sessional, basic C programming structures,		
2	LAB1		Implementation of comments remover of C program in C programming language	A-1	E-1

3	LAB2		Implementation of identifier and number checker of C program in C	A-2, R-1	E-2
4	LAB3		Implementation of tokenizer by using C programming language	A-3, R-2	E-3
5	L2		Basic compiler construction tools (LEX, YACC etc.) Discussion about a simple project for compiler construction	Project, R-3	
5	LAB4		Implementation of Lexical analyzer using by LEX tool	A-4	
6	LAB5		Implementation of Syntax analyzer using by YACC tool Discussion Class for Mid-term Exam	A-5, R-4	
			Mid-term Exam		
7	MT		Exam based on LAB1, LAB2, LAB3, LAB4, LAB5	Viva, Report	E-4 (Exam)
8	LAB6		Implementation of FIRST and FOLLOW of non-terminals from a CFG grammar	A-6, R-5	E-5
9	LAB7		Implementation of Left recursion remover in C	A-7, R-6	
10	LAB7		Implementation of left factoring remover in C	A-7, R-6	E-6
11	LAB8		Implementation of Predictive Parsing Table using FIRST and FOLLOW from a grammar	A-8, R-7	E-7
12	LAB9		Implementation of Shift-reduce Parser from a CFG	A-9, R-8	E-8
13	L4		Construction of Calculator using LEX and YACC	Project, R9	E-9 (Project)
14	LAB10		Implementation of intermediate code generation using C programming	A-10, R-10	E-10
15	L4		Lab report and project submission. Discussion and reviews for final examination		
Final Examination (November-December 2025)				Saifur Rahaman Assistant Professor, CSE, IIUC	

Lab Requirement:

Hardware: PC [with latest Configuration]

Software: OS: Windows 7 or above, C compiler, Linux OS, YACC and LEX tools

Lab Instructions for Student:

- Students should report to the concerned lab as per the time table.
- After completion of the lab program, certification of the concerned lab instructor in the observation book is necessary.
- Student should bring a notebook and should enter the readings observations into the notebook while performing the experiment.
- The record of observations along with the detailed experimental procedure of the experiment in the immediate last session should be submitted and certified staff member in-charge.
- Students should be present in the labs for total scheduled duration.
- Students are required to prepare thoroughly to perform the experiment before coming to laboratory.

Text Books:

- 1) **Compilers, Principles Techniques and Tools**, Alfred V Aho, Ravi Sethi, Jeffrey D. Ullman, 2nd ed, Pearson, 2005

Recommended Books:

1. **Compiler Construction theory and practice**, William A Barret, R. M. Bates, 2nd Ed.
2. **Compiler Design in C**, A.J Holub
3. ***Flex and Bison: Text Processing Tools***, Levine. O'Reilly Media. 2009. ISBN: 0596155972